PySpark Operations : Structured & Deep Notes

What Have We Covered So Far

- Transition from **RDD → DataFrame**
- Why schema matters
- Declarative vs imperative programming
- Lazy evaluation
- Logical plan → Physical plan → Execution
- Catalyst optimizer

Now we move into:

> **How we actually manipulate and operate on data in PySpark**

What Do Operations Mean in PySpark?

In PySpark, **operations** define how data should be processed.

They:

- Describe transformations on DataFrames or RDDs
- Do NOT execute immediately
- Build an internal execution plan

- Run only when triggered by an action

Operations are the foundation of:

- Filtering
- Joining
- Aggregating
- Reshaping
- Writing data

🔎 Analogy

Planning a train journey:

- Choosing stations = filtering
- Selecting seat = selecting columns
- Changing trains = joining datasets
- Booking ticket = action

Nothing moves until the ticket is booked.

Transformations vs Actions (High-Level)

Aspect	Transformations	Actions
Purpose	Define processing logic	Trigger execution
Execution	Lazy	Immediate
Output	New DataFrame/RDD	Value or side-effect
Optimization	Can be optimized	Cannot be optimized further

Resource Use	No execution yet	Uses cluster resources

🔎 Enterprise Analogy

In an enterprise pipeline:

- Cleaning, filtering, enriching → Transformations
- Running dashboard/report → Action

Spark executes everything in one optimized run when an action occurs.

Lazy Evaluation : Why Operations Don’t Run Immediately

Spark:

- Records transformations
- Builds a logical plan
- Waits until an action is called

Why?

Because lazy evaluation allows:

- Global optimization
- Reduced data scans
- Reduced shuffles
- Better resource usage

- Fault tolerance via lineage

🔎 Cloud Data Warehouse Analogy

Creating SQL views does not process data.
Data runs only when a report queries the view.

Same in Spark.

Common Misunderstandings

- “filter() already executed” ❌
- “Performance issue caused by select() line” ❌
- Overusing show() during development ❌

Lineage & Logical Execution Flow

Lineage = History of transformations.

Spark tracks:

- Source dataset
- All transformations applied
- Dependency chain

If a partition fails:

- Spark recomputes using lineage

🔎 ETL Analogy

Raw → Clean → Enrich → Aggregate

Spark doesn’t store all intermediate steps by default.
It stores how to recreate them.

How Operations Form Execution Plan

Architecture Flow:

User Code
↓
Transformations
↓
Logical Plan
↓
Catalyst Optimizer
↓
Physical Plan
↓
Action
↓
Cluster Execution

Catalyst Optimizer

Catalyst:

- Optimizes logical plans
- Rewrites execution strategy
- Reduces computation cost

What Catalyst Optimizes

- Predicate pushdown
- Projection pruning

- Join reordering
- Expression simplification
- Shuffle reduction

🔎 SQL Engine Analogy

Like BigQuery/Snowflake query optimizer rewriting SQL internally.

Transformation Operations

Core Transformations

- select / selectExpr
- withColumn
- drop / rename
- filter / where
- distinct
- dropDuplicates

These build the logical plan.

Row-Level vs Column-Level Operations

Row-Level Operations

Operate record-by-record.

Examples:

```
df.filter(col("amount") > 1000)
```

Used for:

- Filtering
- Conditional checks
- Validations

Column-Level Operations

Operate at column abstraction level.

Example:

```
df.withColumn("amount_usd", col("amount") * 1.1)
```

Used for:

- Derived columns
- Business metrics
- Feature engineering

🔎 Analogy

Row-level = Checking each customer manually
Column-level = Creating a new business attribute across all customers

Column-level aligns better with Spark's optimization engine.

Conditional Logic

when / otherwise

```
df.withColumn(
    "risk_level",
    when(col("amount") > 5000, "HIGH")
    .when(col("amount") > 2000, "MEDIUM")
    .otherwise("LOW")
)
```

Used for:

- Segmentation
- Classification
- Risk modeling

Null Handling

Real-world data always contains nulls.

Functions:

- isNull()
- isNotNull()
- na.fill()
- na.drop()

Example:

```
df.na.fill({"country": "UNKNOWN"})
```

Important for:

- Data quality
- Aggregation safety
- Join correctness

Aggregation Operations

Aggregations summarize data.

Basic Example

```
df.groupBy("store_id").agg(
    sum("amount"),
    avg("amount"),
    count("*")
)
```

Why Aggregations Are Expensive

They require:

- Shuffle
- Network I/O
- Disk spill
- Task coordination

🔎 Analogy

Like collecting sales from all regional offices to compute national totals.

Data must move centrally.

Production Tip

Always:

- Filter before aggregation
- Reduce data size early
- Avoid high-cardinality grouping keys

Shuffle : Critical Concept

Shuffle occurs when:

- groupBy
- join
- distinct
- repartition

Shuffles are:

- Network heavy
- Disk heavy
- Expensive

Most performance issues come from excessive shuffles.

Join Operations

Joins combine datasets.

Example:

```
orders_df.join(customers_df, on="customer_id", how="inner")
```

Join Types

Join Type	Purpose
Inner	Matching records only
Left	Preserve left dataset
Right	Preserve right dataset
Full	Preserve both
Left Semi	Existence check
Left Anti	Find unmatched

🔎 Enterprise Analogy

Customer table + Orders table = Business insights.

Join Keys vs Join Conditions

Join Keys

Simple equality:

```
on="customer_id"
```

Join Conditions

Custom logic:

```
orders_df["cust_id"] == customers_df["customer_id"]
```

Use conditions when:

- Column names differ
- Multiple conditions required

Broadcast Joins

Broadcast = Send small dataset to all executors.

```
from pyspark.sql.functions import broadcast
df.join(broadcast(dim_df), "id")
```

Why Faster?

- Avoids large shuffle
- Local join execution
- Reduced network traffic

When NOT to Use

- Dataset is not small
- Executor memory limited

Regular vs Broadcast Join

Aspect	Regular	Broadcast
Shuffle	Yes	No
Network Cost	High	Low
Best For	Large-large	Large-small

Accumulators

Used for:

- Counting errors
- Tracking metrics
- Debugging distributed jobs

Executors can increment.
Driver reads value.

Example:

```
invalid_count = sc.accumulator(0)
```

Broadcast Variables

Used to share:

- Lookup tables
- Reference datasets
- Small dictionaries

```
broadcast_var = sc.broadcast(data)
```

Read-only on executors.

Accumulator vs Broadcast

Feature	Accumulator	Broadcast
Purpose	Metrics	Reference data
Writable	Yes (increment only)	No
Read by	Driver	Executors

Data Reshaping Operations

Used to change structure.

Includes:

- explode
- pivot
- unpivot
- stack

---

explode()

Used to flatten arrays.

```
df.withColumn("letter", explode("letters"))
```

---

pivot()

Convert rows → columns.

```
df.groupBy("store").pivot("product").sum("sales")
```

---

unpivot (stack)

Convert columns → rows.

```
selectExpr("stack(...)")
```

UDF – User Defined Function

Used when:

- Built-in functions insufficient
- Custom business logic required
- External library needed

How Spark Treats UDF

- Black box
- Cannot optimize
- Executed row-by-row
- Slower than built-in functions

Why UDFs Are Expensive

- Break Catalyst optimization

- Prevent predicate pushdown
- Increase CPU overhead
- Reduce execution efficiency

Rule

Use UDF only when:

> Built-in functions cannot solve the problem.

Actions : What Actually Triggers Execution

Common Actions:

- show()
- count()
- collect()
- take()
- first()
- foreach()
- write()

Most Important Production Action

```
df.write.parquet("/path")
```

Write is:

- Execution trigger
- Job boundary
- Materialization point

Performance-Oriented Operations

cache() vs persist()

Cache:

```
df.cache()
```

Persist:

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Use when:

- Data reused multiple times

Avoid when:

- Used only once

Repartition vs Coalesce

Operation	Shuffle	Use Case

repartition	Yes	Rebalance data
coalesce	No	Reduce partitions

Repartition = Expensive but balanced
Coalesce = Cheap but uneven

Avoiding Unnecessary Shuffles

Common Mistakes:

- Repartitioning without reason
- Aggregating before filtering
- Joining without pruning columns
- High cardinality groupBy

Always:

Filter → Select → Join → Aggregate

Writing Data (Sink Operations)

Writing is:

- Final stage
- Most critical in production
- Action that triggers execution

Common Formats

- Parquet (preferred)
- ORC
- JSON
- CSV

Write Modes

Mode	Behavior
overwrite	Replace data
append	Add data
error	Fail if exists
ignore	Skip if exists

Be careful with overwrite in production.

Partitioned Writing

```
df.write.partitionBy("country", "year")
```

Improves:

- Query speed
- Downstream filtering
- Manageability

Data Skewness

Data skew = uneven key distribution.

Example:

- One customer has 50% of data
- One product extremely popular

Effects

- Long-running tasks
- Executor OOM
- Repeated retries
- Job instability

OOM Causes

- Large partitions
- Large aggregation state
- Excessive shuffle spill

Reading Execution Plans (explain)

```
df.explain()
```

Shows:

- Logical plan
- Physical plan
- Join strategy
- Shuffle operations

Why Execution Plans Matter

Without reading plan:

- You may trigger shuffle accidentally
- Join may not be broadcast
- Filter may not push down
- Cache may not be used

Execution plans provide:

- Visibility
- Control
- Performance tuning insight

🎯 Key Interview Questions

- What is lazy evaluation?
- What triggers execution in Spark?

- Why are aggregations expensive?
- Explain shuffle in Spark.
- Difference between repartition and coalesce?
- When should you use broadcast join?
- Why are UDFs slow?
- What causes data skew?
- How does Catalyst optimize queries?
- When should you cache data?

🔁 Final Recap

- Transformations build logic.
- Actions trigger execution.
- Lazy evaluation enables global optimization.
- Shuffles are expensive — minimize them.
- Aggregations & joins dominate runtime.
- Broadcast joins optimize large–small joins.
- UDFs are powerful but expensive.
- Partitioning & caching require discipline.
- explain() is your debugging tool.